

02_API_STANDARDS

- [SUOP ERP v1.0 — Enterprise API Standards](#)
 - [1. Purpose](#)
 - [2. REST Conventions](#)
 - [3. URL Naming](#)
 - [4. Versioning](#)
 - [5. Pagination](#)
 - [6. Filtering](#)
 - [7. Sorting](#)
 - [8. Bulk APIs](#)
 - [9. Idempotency](#)
 - [10. Error Codes](#)
 - [11. Response Standards](#)
 - [12. Authentication](#)
 - [13. Authorization](#)
 - [14. Rate Limiting](#)
 - [15. API Documentation Strategy](#)
 - [16. Request/Response Logging](#)
 - [17. CORS](#)
 - [18. API Security](#)
 - [19. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise API Standards

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval

Field	Value
Date	2026-07-11
Approval Required	Project Owner

1. Purpose

Every API endpoint in SUOP ERP — internal or external, present or future — must conform to this document. The standards ensure that any developer (internal or third-party) can predict how an endpoint behaves without reading its implementation.

Hard rule: No endpoint ships without conforming to these standards. PR review enforces this. CI runs contract tests against every endpoint.

2. REST Conventions

2.1 HTTP Methods

Method	Intent	Idempotent	Safe	Body
GET	Read resource(s)	Yes	Yes	No
POST	Create resource / trigger action	No	No	Yes
PUT	Replace entire resource	Yes	No	Yes
PATCH	Partial update	No	No	Yes (JSON Patch or partial)
DELETE	Soft delete resource	Yes	No	No
HEAD	Check existence (no body)	Yes	Yes	No

Method	Intent	Idempotent	Safe	Body
OPTIONS	CORS preflight	Yes	Yes	No

Rules: - GET must never mutate state. No side effects. No ? action=delete query params. - POST returns 201 Created with Location header pointing to the new resource. - PUT replaces the entire resource — omitted fields reset to defaults (or rejected if required). - PATCH accepts partial updates — only provided fields change. - DELETE is always soft delete (sets deleted_at). Returns 204 No Content . Hard delete is admin-only via separate endpoint.

2.2 HTTP Status Codes

Code	Meaning	When to use
200 OK	Success	GET, PATCH, PUT, action endpoints
201 Created	Resource created	POST that creates new resource
202 Accepted	Request accepted, processing async	Long-running operations (e.g., generate COA PDF)
204 No Content	Success, no body	DELETE, action with no return value
400 Bad Request	Client error — malformed request	Invalid JSON, missing required field
401 Unauthorized	Not authenticated	Missing/invalid JWT
403 Forbidden	Authenticated but not authorized	Missing permission
404 Not Found	Resource does not exist	Invalid ID, soft-deleted record

Code	Meaning	When to use
405 Method Not Allowed	HTTP method not supported	POST to a GET-only endpoint
409 Conflict	State conflict	Optimistic lock version mismatch, duplicate unique
410 Gone	Resource permanently removed	Hard-deleted record
412 Precondition Failed	ETag/If-Match failed	Conditional request failed
422 Unprocessable Entity	Valid JSON, business rule violation	Supplier not active, budget exceeded
429 Too Many Requests	Rate limited	Returned with <code>Retry-After</code> header
500 Internal Server Error	Server error	Unhandled exception
502 Bad Gateway	Upstream service failed	S3 down, email service down
503 Service Unavailable	Temporary outage	Maintenance mode, deploys
504 Gateway Timeout	Upstream timeout	External service timeout

Forbidden: `200 OK` for errors. Always use the correct 4xx/5xx code.

2.3 Safe Methods & Caching

- `GET` and `HEAD` are cacheable
- Responses include `ETag` (hash of response body) and `Last-Modified` headers

- Clients may send `If-None-Match` / `If-Modified-Since` → server returns `304 Not Modified` if unchanged
 - `Cache-Control: private, max-age=60` for short-lived caches (list endpoints)
 - `Cache-Control: no-store` for sensitive data (audit logs, user lists)
-

3. URL Naming

3.1 URL Structure

```
/api/v1/<resource>/<id>/<sub-resource>/<sub-id>?<query-params>
```

- `/api` — fixed prefix for all API routes
- `/v1` — version (see §4)
- `<resource>` — plural kebab-case noun (e.g., `purchase-orders`, `goods-receipts`)
- `<id>` — UUID v7
- `<sub-resource>` — plural kebab-case (e.g., `/purchase-orders/{id}/lines`)
- `<query-params>` — filtering, sorting, pagination (see §§6-8)

3.2 Resource Naming Rules

- **Always plural nouns:** `/purchase-orders` not `/purchase-order`
- **Always kebab-case:** `/goods-receipts` not `/goodsReceipts` or `/goods_receipts`
- **Never verbs in URL:** Use HTTP method to express action. `/purchase-orders/{id}/cancel` is forbidden — use `POST /purchase-orders/{id}/transitions` with `{target: 'CANCELLED'}`.
 - **Exception:** Workflow transitions and business actions that don't fit CRUD use a sub-resource verb: `POST /purchase-orders/{id}/submit`, `POST /purchase-orders/{id}/approve`. These are explicit and documented.

- **Never expose DB internals:** No `/tables/users` or `/queries/run-q-123`. URLs reflect business domain, not schema.
- **ID format:** UUID v7 only. No sequential IDs in URLs (security + scale).

3.3 URL Examples

```
GET    /api/v1/products
GET    /api/v1/products/{id}
POST   /api/v1/products
PATCH /api/v1/products/{id}
DELETE /api/v1/products/{id}

GET    /api/v1/products/{id}/barcodes
POST   /api/v1/products/{id}/barcodes

GET    /api/v1/purchase-orders/{id}/lines
POST   /api/v1/purchase-orders/{id}/lines
PATCH /api/v1/purchase-orders/{id}/lines/{lineId}

POST   /api/v1/purchase-orders/{id}/submit
POST   /api/v1/purchase-orders/{id}/approve
POST   /api/v1/purchase-orders/{id}/reject
POST   /api/v1/purchase-orders/{id}/cancel

GET    /api/v1/products/{id}/audit-trail
GET    /api/v1/products/{id}/workflows/available-transitions
```

3.4 Special Endpoints

- `GET /api/v1/_internal/health` — health check (no auth)
- `GET /api/v1/_internal/ready` — readiness check (no auth, checks DB/Redis)
- `GET /api/v1/_internal/metrics` — Prometheus metrics (auth: API key)

- `GET /api/v1/_internal/schemas/{name}` — fetch zod schema for codegen
- `GET /api/v1/_internal/jobs` — admin job inspection
- `GET /api/v1/reference/{type}` — reference data (no auth, cached)

The `_internal` prefix is reserved for platform endpoints, never business resources.

4. Versioning

4.1 Version in URL

```
/api/v1/purchase-orders
/api/v2/purchase-orders (future, if breaking change
needed)
```

- Version is a major version only (v1, v2 — never v1.5)
- Minor changes (additive — new field, new endpoint) do NOT bump version
- Major changes (breaking — removed field, changed type, changed semantics) bump version

4.2 Backward Compatibility

- Old version maintained for **6 months** after new version ships
- Deprecation notice in response header: `Sunset: Sat, 31 Jan 2027 00:00:00 GMT`
- Deprecation notice in API docs and migration guide
- After 6 months, old version returns `410 Gone` with migration instructions

4.3 What Counts as Breaking

Change	Breaking?
Add new endpoint	No

Change	Breaking?
Add new optional field to request body	No
Add new field to response body	No
Remove field from response body	Yes
Change field type	Yes
Change field semantics (e.g., <code>status</code> enum value meaning)	Yes
Change default value of optional field	Yes
Change error response shape	Yes
Change authentication scheme	Yes
Add new required field to request body	Yes

4.4 No Version in Header

URL versioning chosen over header versioning (`Accept: application/vnd.suop.v1+json`): - URL versioning is visible in logs, monitoring, debugging - Easier to route at CDN/load balancer - Easier to test in browser/Postman without setting headers - Trade-off: URL changes are visible to users — acceptable

5. Pagination

5.1 Two Pagination Styles

Style 1 — Offset pagination (for admin tables, simple lists):

```
GET /api/v1/products?page=1&pageSize=25
```

- Use for: small result sets, jump-to-page UI
- Limit: `pageSize` max 200; `page` max 1000 (use cursor for deeper)

- Response includes `meta.total`, `meta.totalPages`, `meta.hasNext`, `meta.hasPrev`

Style 2 — Cursor pagination (for infinite scroll, large datasets, audit logs):

```
GET /api/v1/audit-logs?
cursor=eyJpZCI6IjAxMjM...&pageSize=25
```

- Cursor is base64-encoded JSON of the last row's sort key (e.g., `{id: '...', createdAt: '...'}`)
- Use for: tables >100k rows, infinite scroll UI, append-mostly data (audit logs, stock ledger)
- Response includes `meta.nextCursor` (null if no more pages)
- Cannot jump to arbitrary page — only next/prev

5.2 Default Page Size

- Default: `pageSize=25`
- Max: `pageSize=200` (returns `400` if exceeded)
- Min: `pageSize=1` (returns `400` if `pageSize=0`)

5.3 Pagination Response Shape

```
{
  "success": true,
  "data": [...],
  "meta": {
    "page": 1,
    "pageSize": 25,
    "total": 482,
    "totalPages": 20,
    "hasNext": true,
    "hasPrev": false,
    "correlationId": "01HXY..."
  }
}
```

```
}  
}
```

For cursor pagination:

```
{  
  "success": true,  
  "data": [...],  
  "meta": {  
    "pageSize": 25,  
    "nextCursor": "eyJpZCI6IjAx...",  
    "hasNext": true,  
    "correlationId": "01HXY..."  
  }  
}
```

6. Filtering

6.1 Filter Syntax

Filters are query parameters with operator suffixes:

```
GET /api/v1/purchase-orders?  
status=APPROVED&supplierId=abc-123&totalValue.gte=10000
```

6.2 Operators

Operator	Suffix	Example	SQL equivalent
Equals	(none)	status=APPROVED	status = 'APPROVED'
	.ne	status.ne=DRAFT	

Operator	Suffix	Example	SQL equivalent
Not equals			<code>status != 'DRAFT'</code>
In	<code>.in</code>	<code>status.in=APPROVED,ORDERED</code>	<code>status IN ('APPROVED','ORDERED')</code>
Not in	<code>.nin</code>	<code>status.nin=DRAFT,CANCELLED</code>	<code>status NOT IN (...)</code>
Greater than	<code>.gt</code>	<code>totalValue.gt=1000</code>	<code>total_value > 1000</code>
Greater/ equal	<code>.gte</code>	<code>totalValue.gte=1000</code>	<code>total_value >= 1000</code>
Less than	<code>.lt</code>	<code>totalValue.lt=10000</code>	<code>total_value < 10000</code>
Less/ equal	<code>.lte</code>	<code>totalValue.lte=10000</code>	<code>total_value <= 10000</code>
Between	<code>.between</code>	<code>createdBetween=2026-01-01,2026-06-30</code>	<code>BETWEEN</code>
Contains	<code>.contains</code>	<code>name.contains=kaju</code>	<code>ILIKE '%kaju%'</code>
Starts with	<code>.startsWith</code>	<code>sku.startsWith=KK-</code>	<code>ILIKE 'KK'</code>
Ends with	<code>.endsWith</code>	<code>sku.endsWith=-250</code>	<code>ILIKE '%-250'</code>
Is null	<code>.isNull</code>	<code>deletedAt.isNull=true</code>	<code>deleted_at IS NULL</code>
Is not null	<code>.isNotNull</code>	<code>approvedAt.isNotNull=true</code>	<code>approved_at IS NOT NULL</code>

6.3 Multiple Filters (AND)

Multiple filter params are AND-ed:

```
GET /api/v1/products?  
status=ACTIVE&category=SWEETS&stock.lt=100  
→ status = 'ACTIVE' AND category = 'SWEETS' AND stock < 100
```

6.4 OR Filters

For OR conditions, use the `filter` parameter with a JSON-encoded filter object (advanced):

```
GET /api/v1/products?filter={"OR":[{"status":"ACTIVE"},  
{"status":"DRAFT"}]}
```

This is URL-encoded in practice. Most use cases use AND via simple query params.

6.5 Date Filters

- Dates in ISO 8601 format: `2026-07-11` or `2026-07-11T14:30:00Z`
- Date ranges: `createdAt.between=2026-01-01,2026-06-30`
- All dates stored in UTC; queries interpret naive dates as UTC

6.6 Filter Validation

- Unknown filter fields return `400` with `{field: 'status', message: 'Unknown field'}`
 - Invalid filter values return `400` (e.g., `createdAt.gt=not-a-date`)
 - Filter operators on incompatible types return `400` (e.g., `status.gt=ACTIVE`)
-

7. Sorting

7.1 Sort Syntax

```
GET /api/v1/purchase-orders?sort=-createdAt,supplierName
```

- Prefix `-` for descending (e.g., `-createdAt` = newest first)
- Multiple sort fields comma-separated (priority order)
- Default sort: `-createdAt` (newest first) if no `sort` param

7.2 Sortable Fields

- Each endpoint declares sortable fields in its OpenAPI spec
- Sorting by non-sortable field returns `400`
- Sorting by sensitive field (e.g., `password_hash`) is forbidden — field not in whitelist

7.3 Sort Stability

- Secondary sort by `id` always applied to ensure stable pagination
- E.g., `sort=-createdAt` becomes `ORDER BY created_at DESC, id ASC`

8. Bulk APIs

8.1 Bulk Create

```
POST /api/v1/products/bulk
Content-Type: application/json

{
  "items": [
    { "name": "Kaju Katli 250g", "sku": "KK-250", ... },
    { "name": "Kaju Katli 500g", "sku": "KK-500", ... }
```

```
]
}
```

- Max 100 items per bulk request
- Returns `207 Multi-Status` with per-item result:

```
{
  "success": true,
  "data": {
    "succeeded": [{ "index": 0, "id": "...", "sku":
      "KK-250" }],
    "failed": [{ "index": 1, "errors": [{ "field": "sku",
      "message": "Already exists" }] }],
    "summary": { "total": 2, "succeeded": 1, "failed": 1 }
  }
}
```

- All-or-nothing mode via `?atomic=true` — fails entire batch if any item invalid

8.2 Bulk Update

```
PATCH /api/v1/products/bulk
{
  "updates": [
    { "id": "...", "version": 1, "data": { "status":
      "DISCONTINUED" } },
    { "id": "...", "version": 3, "data": { "status":
      "DISCONTINUED" } }
  ]
}
```

- Each update must include `version` for optimistic locking
- Returns `207 Multi-Status`

8.3 Bulk Delete

```
DELETE /api/v1/products/bulk
{
  "ids": ["...", "...", "..."]
}
```

- Max 100 IDs per request
- Returns 207 Multi-Status

8.4 Bulk Import (CSV/Excel)

```
POST /api/v1/products/import
Content-Type: multipart/form-data

file: products.csv
```

- Async — returns 202 Accepted with job ID
- Client polls GET /api/v1/_internal/jobs/{jobId} for status
- Job result includes row-level errors
- Max 10,000 rows per import; larger batches via S3 upload + signed URL

8.5 Bulk Export

```
GET /api/v1/products/export?format=csv&filter=...
```

- Async — returns 202 Accepted with job ID
 - Job produces file in S3; client receives signed URL when ready
 - Max 1 million rows per export
-

9. Idempotency

9.1 Idempotency Key

For all `POST` and `PATCH` requests that create or mutate state, clients SHOULD send an idempotency key:

```
POST /api/v1/purchase-orders
Idempotency-Key: 7d8b2c1e-9f3a-4b6c-8e2d-1a5f9b3c7e8d
Content-Type: application/json

{ ... }
```

9.2 Server Behavior

- Server stores (`idempotencyKey`, `tenantId`, `requestHash`, `response`, `expiresAt`) in `idempotency_keys` table
- On retry with same key + same request hash: returns stored response (status + body)
- On retry with same key + different request hash: returns `409 Conflict` (idempotency key reuse)
- Keys expire after 24 hours (configurable)
- Keys are scoped to tenant — same key in different tenants are independent

9.3 Required for

- All `POST` to collection endpoints (create)
- All `POST` to action endpoints (submit, approve, etc.)
- All bulk operations
- All payment/financial mutations

9.4 Optional for

- `GET` (always idempotent by nature)
- `DELETE` (idempotent — soft-deleting already-deleted is no-op)

- `PATCH` (idempotent if client includes `version` for optimistic locking)

10. Error Codes

10.1 Error Code Format

Error codes are namespaced strings:

```
<DOMAIN>.<ENTITY>.<RULE>
```

Examples:

```
VALIDATION.REQUIRED_FIELD
VALIDATION.INVALID_FORMAT
VALIDATION.OUT_OF_RANGE
AUTH.TOKEN_EXPIRED
AUTH.INVALID_CREDENTIALS
AUTH.ACCOUNT_LOCKED
RBAC.PERMISSION_DENIED
PO.SUPPLIER_INACTIVE
PO.BUDGET_EXCEEDED
PO.DUPLICATE_LINE_ITEM
GRN.QUANTITY_EXCEEDS_PO_LINE
STOCK.INSUFFICIENT_STOCK
STOCK.NEGATIVE_BALANCE_FORBIDDEN
CONCURRENCY.VERSION_MISMATCH
CONCURRENCY.DEADLOCK_RETRY
RATE_LIMIT.TOO_MANY_REQUESTS
EXTERNAL.S3_UNAVAILABLE
EXTERNAL.EMAIL_SEND_FAILED
```

10.2 Error Code Registry

All error codes catalogued in `packages/shared/src/enums/error-code.enum.ts`. New codes require PR review. Frontend uses codes to render localized messages.

10.3 Error Response Shape

```
{
  "success": false,
  "error": {
    "code": "PO.SUPPLIER_INACTIVE",
    "message":
      "Supplier SUP-001 is not active and cannot be added
      to a purchase order",
    "details": [
      {
        "field": "supplierId",
        "code": "PO.SUPPLIER_INACTIVE",
        "message": "Supplier status is BLACKLISTED",
        "metadata": { "supplierId": "...", "currentStatus":
          "BLACKLISTED" }
      }
    ],
    "entityId": "po-abc-123",
    "retryAfter": null
  },
  "meta": {
    "correlationId": "01HXY..."
  }
}
```

10.4 Validation Error Shape

For `400 Bad Request` from zod validation:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION.REQUIRED_FIELD",
    "message": "Request body validation failed",
    "details": [
      { "field": "poLines[0].quantity", "code": "REQUIRED",
        "message": "Quantity is required" },
      { "field": "poLines[1].unitPrice", "code":
        "INVALID_FORMAT", "message": "Must be a positive
        number", "metadata": { "actual": -5 } }
    ]
  },
  "meta": { "correlationId": "01HXY..." }
}
```

10.5 Field Paths

Field paths use dot + bracket notation: - `name` - `address.city` -
`poLines[0].quantity` - `poLines[2].taxes.cgst.rate`

11. Response Standards

11.1 Success Envelope

```
{
  "success": true,
  "data": { ... },
  "message": "Purchase order created successfully", //
    optional
  "meta": {
    "correlationId": "01HXY...",
    "page": 1, // only for lists
    "pageSize": 25,
    "total": 482,
    "totalPages": 20,
  }
}
```

```
    "hasNext": true
  }
}
```

11.2 Error Envelope

See §10.3.

11.3 Field Naming in Responses

- **camelCase** in JSON responses (JavaScript convention)
- Database is snake_case; transformation happens in controller layer
- Example: DB column `created_at` → JSON field `createdAt`
- Prisma Client handles this transformation via the `@map` annotation

11.4 Date Format

- All dates in ISO 8601 with timezone: `2026-07-11T14:30:00Z`
- All dates in UTC (Z suffix) — client converts to local timezone for display
- Date-only fields: `2026-07-11` (no time component)
- Duration fields: ISO 8601 duration (`PT5M30S` for 5 min 30 sec)

11.5 Money Fields

- Always returned as objects with amount + currency:

```
{
  "totalValue": {
    "amount": "12500.00",
    "currency": "INR"
  }
}
```

- Amount is **string** (not number) to avoid floating-point precision loss
- Currency is ISO 4217 3-letter code

11.6 Quantity Fields

- Decimal quantities as **string**: "125.500" (preserves precision)
- Integer counts as **number**: 125

11.7 Empty Responses

- 204 No Content — no body (DELETE, action with no return)
- 200 OK with data: null — explicit null when resource exists but has no representation (rare)
- 200 OK with data: [] — empty list (different from null — list exists, has 0 items)

11.8 Sparse Fieldsets

Clients can request specific fields to reduce payload:

```
GET /api/v1/products?fields=id,name,sku,stock
```

- Returns only requested fields
- id always included
- Unknown fields return 400
- Default (no fields param) returns all fields per entity's response schema

12. Authentication

12.1 Authentication Scheme

JWT Bearer tokens:

```
Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

12.2 JWT Structure

- **Algorithm:** HS256 (HMAC-SHA256) — symmetric, simple
- **Future:** RS256 (asymmetric) when external clients need to verify tokens without sharing secret
- **Header:**

```
{ "alg": "HS256", "typ": "JWT", "kid": "v1" }
```

- **Payload:**

```
{  
  "sub": "user-uuid-v7",  
  "tenantId": "tenant-uuid-v7",  
  "email": "rajesh@sudhamrit.com",  
  "roles": ["quality_manager"],  
  "permissions": ["po:read", "po:create"],  
  "iat": 1720000000,  
  "exp": 1720000900, // 15 min from issue  
  "iss": "suop-erp",  
  "aud": "suop-frontend",  
  "jti": "token-uuid-v7"  
}
```

12.3 Token Lifecycle

Token	TTL	Storage
Access token	15 minutes	In-memory (frontend); never localStorage
Refresh token	30 days	httpOnly secure cookie (frontend); hashed in DB
Device session	90 days	

Token	TTL	Storage
		DB (device_sessions table)
API key	1 year (rotated)	DB (api_keys table, hashed)

12.4 Refresh Flow

1. Client makes request with access token
2. If access token expired (401), client calls `POST /api/v1/auth/refresh` with refresh token cookie
3. Server validates refresh token against DB (revocation list), issues new access token + new refresh token (rotation)
4. Old refresh token immediately invalidated (replay detection)
5. New refresh token in httpOnly cookie

12.5 Token Revocation

- **Access tokens:** Cannot be revoked (short-lived — wait 15 min). Mitigated by short TTL.
- **Refresh tokens:** Revocable via `POST /api/v1/auth/logout` (deletes from DB) or admin force-logout
- **JTI blocklist:** Compromised access tokens' JTIs added to Redis blocklist (TTL = remaining token TTL); checked on every request

12.6 Multi-Factor Authentication (MFA)

- **TOTP-based** (Google Authenticator, Authy)
- **Required for:** Admin roles, finance roles, anyone with `system:*` permissions
- **Optional for:** Other users (configurable per tenant)
- **Flow:**
 1. User logs in with email + password → server returns `MFA_REQUIRED` response with `mfaSessionToken`

2. User enters 6-digit TOTP code → `POST /api/v1/auth/mfa/verify` with `mfaSessionToken` + code
3. Server verifies TOTP, issues access + refresh tokens

12.7 API Keys (for integrations)

- Long-lived (1 year), rotated
 - Format: `suop_<random_64_chars>`
 - Stored hashed (argon2id) in `api_keys` table
 - Scoped to specific permissions (not full user permissions)
 - Sent via `X-API-Key` header (not Authorization — distinguish from user JWTs)
-

13. Authorization

13.1 Permission Model

RBAC (Role-Based Access Control) with optional ABAC (Attribute-Based) checks.

13.2 Permission Format

```
<resource>:<action>
```

Examples:

```
org:read      org:create      org:update
org:delete
po:read       po:create      po:approve
po:cancel
product:read  product:create product:update
product:delete
audit:read    audit:read:critical
system:tenant:cross
```


13.3 Permission Check Points

Layer 1 — Route-level (middleware):

```
router.post('/purchase-orders',
  requirePermission('po:create'),
  poController.create
)
```

- Blocks unauthorized users before controller executes
- Returns `403` if missing permission

Layer 2 — Row-level (service):

```
async approve(ctx, poId) {
  const po = await poRepo.findByIdOrThrow(ctx, poId)
  // User can only approve POs they're assigned as approver for
  if (po.approverId !== ctx.userId && !
    ctx.hasPermission('po:approve:any')) {
    throw new AuthorizationError('You can only approve POs
      assigned to you')
  }
  // ...
}
```

Layer 3 — Field-level (response shaping): - Some fields visible only to certain roles - E.g., `costPrice` on products visible to `finance:*` but not `sales:*` - Controller strips fields based on user permissions

13.4 Permission Hierarchy

- `system:*` — super-admin (rare; only platform team)
- `tenant:admin` — tenant administrator (full access within tenant)
- `<resource>:*` — full access to a resource (e.g., `po:*`)
- `<resource>:<action>` — specific action
- `<resource>:<action>:<scope>` — scoped action (e.g., `po:approve:any` vs `po:approve:own`)

13.5 Frontend Permission Checks

- `<PermissionGuard require="po:create"><Button>New P0</Button></PermissionGuard>` — hides button if user lacks permission
- Sidebar menu items filtered by permission
- Routes blocked at route level (redirect to 403 page)
- **Note:** Frontend checks are UX only — backend enforces truth. Never trust frontend for security.

14. Rate Limiting

14.1 Rate Limit Tiers

Tier	Limit	Window	Applies to
Public	60 req/min	IP	<code>/api/v1/auth/login</code> , <code>/api/v1/auth/register</code>
Authenticated	600 req/min	tenant + user	All authenticated endpoints
Heavy	30 req/min	tenant + user	Export, import, bulk operations
Read	1200 req/min	tenant + user	GET endpoints (more lenient)

14.2 Rate Limit Algorithm

- **Sliding window** via Redis (more accurate than fixed window)
- Key: `rate_limit:{tier}:{identifier}:{minute_bucket}`
- On each request: increment counter, check against limit, set TTL 60s

14.3 Rate Limit Response

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 23
X-RateLimit-Limit: 600
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1720000923
```

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT.TOO_MANY_REQUESTS",
    "message": "Rate limit exceeded. Retry after 23
seconds.",
    "retryAfter": 23
  },
  "meta": { "correlationId": "01HXY..." }
}
```

14.4 Burst Allowance

- Token bucket allows short bursts above steady-state limit
- E.g., 600/min allows 100-token burst, refilled at 10/sec
- Prevents legitimate batch operations from being throttled

15. API Documentation Strategy

15.1 OpenAPI Specification

- Every endpoint documented in OpenAPI 3.1 spec
- Spec generated from zod schemas via `zod-to-openapi`
- Spec committed to repo at `packages/shared/openapi.yaml`
- Spec served at `GET /api/v1/_internal/openapi.json`

- Swagger UI at `/api/docs` (development only — disabled in production)

15.2 Documentation Requirements per Endpoint

- Summary (one line)
- Description (paragraph)
- Parameters (path, query, header) with types + examples
- Request body schema with example
- Response schemas for each status code (200, 201, 400, 401, 403, 404, 409, 422, 429, 500)
- Error codes specific to this endpoint
- Required permission
- Idempotency: required/optional/N/A
- Rate limit tier
- Sample request (curl)
- Sample response

15.3 SDK Generation

- TypeScript SDK auto-generated from OpenAPI spec via `openapi-typescript-codegen`
- SDK published as `@suop/api-client` npm package (internal)
- Frontend uses SDK instead of raw `fetch` — guarantees type safety
- Mobile app uses same SDK

15.4 API Changelog

- Every PR that adds/changes an endpoint updates `CHANGELOG.md`
- Entry format:

```
## [Unreleased]
### Added
- POST /api/v1/purchase-orders/{id}/submit – submit PO for approval
### Changed
- GET /api/v1/products now returns `upi` field (was previously `upicode`)
```

Deprecated

- GET /api/v1/products?include=barcodes – use GET /api/v1/products/{id}/barcodes instead

Removed

- None

- Release process generates `CHANGELOG.md` entries into versioned docs

15.5 Versioned Documentation

- Each API version has its own documentation snapshot
- `docs.api.suop.com/v1/...` for v1 docs
- `docs.api.suop.com/v2/...` for v2 docs
- Old versions preserved indefinitely (link rot forbidden)

16. Request/Response Logging

16.1 Logged per Request

- Method, path, query params (values redacted for sensitive fields)
- Request body (redacted for sensitive fields: `password`, `token`, `creditCard`)
- User ID, tenant ID (if authenticated)
- Response status, duration, response size
- Correlation ID

16.2 Sensitive Field Redaction

- Field names matching `/password|token|secret|key|auth|credit/i` have values replaced with `[REDACTED]`
- PII fields (`email`, `phone`, `aadhaar`, `pan`) hashed (SHA-256) for correlation without exposure
- Redaction config in `logging/redaction-config.ts`

16.3 Sampling

- 100% of mutating requests (POST/PUT/PATCH/DELETE) logged at INFO
 - 10% of GET requests logged at INFO (90% at DEBUG)
 - 100% of error responses (4xx, 5xx) logged at WARN/ERROR
 - 100% of slow requests (>1s) logged at WARN
-

17. CORS

17.1 Allowed Origins

- Production: `https://app.sudhamrit.com`, `https://erp.sudhamrit.com`
- Staging: `https://staging.sudhamrit.com`
- Development: `http://localhost:3000`, `http://localhost:3001`
- Mobile: capacitor origin (`capacitor://localhost`)

17.2 Headers

```
Access-Control-Allow-Origin: <allowed origin or null>
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE,
OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type,
Idempotency-Key, X-Request-Id
Access-Control-Expose-Headers: X-Request-Id, X-RateLimit-*,
ETag, Location
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 86400
```

17.3 Preflight Caching

- `OPTIONS` preflight cached for 24 hours (`Access-Control-Max-Age: 86400`)
 - Reduces `OPTIONS` traffic to ~1 per origin per day per browser
-

18. API Security

18.1 Input Validation

- All inputs validated by zod schema before controller executes
- Unknown fields rejected (`z.object({...}).strict()`)
- String length limits enforced
- Numeric ranges enforced
- Date format validation

18.2 Output Encoding

- All JSON responses encoded by framework (no manual `JSON.stringify` of user data)
- No HTML responses from API (CSRF protection — see Security doc)
- Headers like `X-Content-Type-Options: nosniff` prevent MIME sniffing

18.3 File Upload Validation

- MIME type whitelist per category
- File size limits per category
- Magic byte verification (not just trusting Content-Type header)
- Virus scan via ClamAV before storage

18.4 SQL Injection Prevention

- Prisma parameterized queries everywhere — no raw SQL with string interpolation
- Raw queries (rare) use parameterized `Prisma.sql` template literal
- Code review enforces no `Prisma.$queryRawUnsafe` ever

18.5 Rate Limit per User (not just IP)

- Rate limits keyed on `userId` (authenticated) or `IP` (unauthenticated)
- Prevents single user from bypassing limits via IP rotation

- Failed login attempts: 10 per user per 15 min → account lockout (30 min)

19. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-A1	Pagination default	Offset for lists, cursor for logs	Cursor everywhere
Q-A2	Filter syntax	Query param operators (<code>.gt</code> , <code>.in</code>)	JSON-encoded filter
Q-A3	Bulk max items	100	50, 500
Q-A4	Idempotency key TTL	24 hours	1 hour, 7 days
Q-A5	JWT algorithm	HS256 (Phase 0) → RS256 (Phase X)	RS256 from start
Q-A6	Access token TTL	15 minutes	5 min, 30 min
Q-A7	Refresh token TTL	30 days	7 days, 90 days
Q-A8	MFA requirement	Required for admin/finance	Required for all
Q-A9	API docs in production	Disabled (internal only)	Enabled (public)
Q-A10	SDK generation	Auto from OpenAPI	Manual

Approval Block

Approved by: _____ **Date:** _____

End of Enterprise API Standards Document